

ACTIVIDAD LABORATORIO NO.2
IMPLEMENTACIÓN DE PROYECCIONES 3D EN OPENGL: ORTOGONAL, GABINETE,
PERSPECTIVA SIMÉTRICA Y PERSPECTIVA OBLICUA

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING ROGERIO ORLANDO BELTRAN CASTRO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
24 DE MAYO
2026

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
DESARROLLO ACTIVIDAD	4
1. Fundamentos de proyección en computación grafica	4
2. El pipeline de transformación en OpenGL	4
3. Descripción matemática de cada proyección	5
a. Proyección Ortogonal	5
b. Proyección Gabinete	5
c. Proyección Perspectiva Simétrica	5
d. Proyección Perspectiva Oblicua	5
4. Arquitectura del Programa.....	6
a. Estructura general	6
b. Patrón de renderizado por cuadrante	6
c. Método init() - Inicialización OpenGL	7
d. Metodo reshape() - Redimensionado	7
e. Método display() - Loop de renderizado	8
f. Funcion dibujarCubo() - Geometria del cubo	8
g. Funciones aplicarRotacion() y dibujarTexto()	9
h. Proyección Ortogonal	10
i. Proyección Gabinete	10
j. Proyección Perspectiva Simétrica	11
k. Proyección Perspectiva Oblicua	12
l. Separadores y Dispose	12
m. Teclado	13
5. Configuración del Entorno en NetBeans	13
a. Librerías del proyecto	14
b. Java Platform Manager	14
c. Archivo project.properties	15
6. Captura de Pantalla del Programa en Ejecución	16
7. Análisis Comparativo de las Proyecciones	17
a. Tabla comparativa.....	17
b. Diferencias observadas	18
8. Funcionamiento y Fallos.....	18
a. Funcionamiento general	18
b. Limitaciones técnicas.....	19
9. Entorno de Desarrollo.....	19
a. Dependencias JAR	19
b. Configuración JVM	20
CONCLUSIONES DE LA ACTIVIDAD	20
BIBLIOGRAFÍA.....	21
AGRADECIMIENTO	22

INTRODUCCIÓN

En el contexto de la informática gráfica y la visualización computacional, OpenGL se ha consolidado como uno de los estándares más importantes para el desarrollo de aplicaciones gráficas interactivas en dos y tres dimensiones. Su arquitectura basada en el pipeline gráfico permite comprender de manera práctica los procesos de transformación geométrica, proyección, rasterización y representación visual de objetos tridimensionales, constituyendo una herramienta fundamental en áreas como el desarrollo de videojuegos, simuladores, sistemas CAD, realidad virtual, visualización científica y motores gráficos en tiempo real. El estudio de OpenGL no solo fortalece las competencias técnicas relacionadas con la programación gráfica, sino que también introduce conceptos matemáticos y computacionales esenciales asociados al manejo de coordenadas espaciales, matrices de transformación y representación visual de escenas tridimensionales.

El presente laboratorio desarrolla la implementación de un sistema de visualización de proyecciones 3D utilizando OpenGL mediante la biblioteca JOGL (Java OpenGL) en lenguaje Java. La aplicación tiene como propósito principal analizar y comparar distintos modelos de proyección tridimensional aplicados sobre un mismo objeto geométrico, permitiendo observar de forma práctica las diferencias matemáticas y visuales entre proyecciones paralelas y perspectivas. Para ello, el programa genera una ventana OpenGL dividida en cuatro cuadrantes independientes, cada uno encargado de representar un cubo tridimensional utilizando una técnica de proyección específica: proyección ortogonal, proyección gabinete, perspectiva simétrica y perspectiva oblicua off-axis.

El objetivo principal de este trabajo consiste en comprender e implementar los fundamentos de las transformaciones de proyección en gráficos computacionales mediante el uso de matrices de transformación y funciones clásicas de OpenGL como `glOrtho` y `glFrustum`. A través del desarrollo del laboratorio se busca fortalecer conceptos relacionados con el pipeline gráfico, sistemas de coordenadas, geometría tridimensional, matrices homogéneas 4x4 y representación espacial en entornos 3D. Asimismo, el proyecto permite analizar cómo diferentes configuraciones del volumen de visión afectan la percepción visual de profundidad, paralelismo y convergencia de líneas dentro de una escena tridimensional.

A diferencia de una implementación básica centrada únicamente en el renderizado geométrico, el programa fue desarrollado bajo un enfoque modular y estructurado, organizando el proceso de renderizado en múltiples funciones especializadas encargadas de configurar cada tipo de proyección, administrar el viewport correspondiente y representar el objeto tridimensional bajo distintas condiciones de visualización. Esta organización responde a principios de ingeniería de software como la separación de responsabilidades, la mantenibilidad del código, la reutilización de componentes y la claridad estructural del sistema, permitiendo además una mejor comprensión del flujo de renderizado implementado por OpenGL.

El desarrollo del laboratorio inicia con la configuración del entorno gráfico utilizando JOGL y Apache NetBeans IDE 28, estableciendo el contexto OpenGL y la ventana principal de renderizado. Posteriormente, se implementa la división de la ventana mediante múltiples viewports, permitiendo representar simultáneamente las diferentes proyecciones sobre un mismo objeto geométrico. En fases posteriores se construyen las matrices de proyección correspondientes a cada técnica, incluyendo el uso de transformaciones ortogonales, matrices de cizallamiento para la proyección gabinete y frustums simétricos y descentrados para las perspectivas. Finalmente, se incorporan transformaciones de rotación y traslación sobre el cubo tridimensional para facilitar la visualización comparativa de las diferencias perceptuales entre cada modelo de proyección.

Adicionalmente, el sistema implementa mecanismos de redimensionamiento dinámico mediante el callback `reshape`, garantizando la correcta actualización de las dimensiones de los viewports cuando la ventana cambia de tamaño. También se emplea renderizado con buffer de profundidad (`depth testing`) para asegurar la correcta visualización espacial de las caras del cubo, demostrando cómo las técnicas de manejo de profundidad y transformación geométrica influyen directamente en la percepción tridimensional de la escena.

Este laboratorio no solo permite comprender el funcionamiento fundamental de las proyecciones en OpenGL clásico, sino que también establece una base sólida para futuros desarrollos relacionados con gráficos computacionales avanzados, motores de renderizado, simulación tridimensional, realidad

virtual y programación gráfica moderna basada en shaders. Al integrar conceptos matemáticos, principios de programación orientada a objetos y técnicas de representación visual, el proyecto evidencia la importancia de la informática gráfica como una de las áreas fundamentales dentro de la ingeniería informática y las ciencias computacionales.

Finalmente, este trabajo representa una aplicación práctica de los conceptos fundamentales de visualización computacional y proyección tridimensional, destacando la relevancia de las matrices de transformación, los modelos de proyección y la estructuración modular del software como elementos esenciales para el desarrollo de aplicaciones gráficas robustas, organizadas y visualmente eficientes.

DESARROLLO ACTIVIDAD

A continuación, se presenta el desarrollo paso a paso de la implementación del sistema de proyecciones tridimensionales utilizando OpenGL mediante la biblioteca JOGL en lenguaje Java, siguiendo un enfoque modular, estructurado y orientado a la separación de responsabilidades. Cada componente del sistema fue desarrollado mediante funciones independientes encargadas de configurar un tipo específico de proyección, administrar su respectivo viewport y renderizar el objeto tridimensional bajo distintas configuraciones geométricas y visuales. Esta organización permitió construir un código más claro, mantenible y escalable, facilitando además la comprensión del pipeline gráfico y del comportamiento matemático de las transformaciones de proyección en entornos tridimensionales.

En las siguientes secciones se documentan detalladamente los procedimientos realizados para la construcción de la ventana OpenGL, la división de la interfaz gráfica en cuadrantes, la implementación de las diferentes matrices de proyección y el renderizado del cubo tridimensional utilizado como objeto de prueba. Asimismo, se describen las primitivas gráficas empleadas, las transformaciones geométricas aplicadas, los cálculos matemáticos relacionados con los frustums de visión y las principales características visuales incorporadas durante el desarrollo del laboratorio, permitiendo analizar comparativamente el comportamiento de las proyecciones ortogonales, gabinete y perspectivas implementadas.

1. Fundamentos de proyección en computación grafica

En el pipeline grafico 3D, la etapa de proyección transforma coordenadas tridimensionales del espacio de vista (eye space) al espacio de clip (clip space), reduciéndolas a una representación bidimensional apta para rasterización. Esta transformación es una aplicación lineal que determina no solo que objetos son visibles, sino como se percibe la profundidad y la forma de la escena.

Existen dos grandes familias de proyección: la proyección paralela, en la que los rayos visuales son paralelos entre si (punto de vista en el infinito), y la proyección perspectiva, en la que los rayos convergen en el centro de proyección (ojo del observador). Ambas familias se implementan en OpenGL mediante matrices 4x4 que actúan sobre coordenadas homogéneas.

2. El pipeline de transformación en OpenGL

OpenGL legacy utiliza dos matrices principales en el pipeline de transformación geométrica:

- **GL_PROJECTION:** transforma coordenadas de vista al volumen de vista canónico normalizado (NDC). Define el tipo y parámetros de la proyección.
- **GL_MODELVIEW:** transforma coordenadas de objeto a coordenadas de vista. Combina la transformación de modelo con la de cámara.

```
v_clip = M_projection * M_modelview * v_object
```

Donde:

v_object = vértice en coordenadas de objeto (homogéneas)

M_modelview = GL_MODELVIEW (rotación + traslación)

M_projection= GL_PROJECTION (tipo de proyección)

v_clip = vértice en clip space -> NDC -> pantalla

3. Descripción matemática de cada proyección

Para comprender el funcionamiento de las diferentes técnicas de visualización implementadas en el laboratorio, resulta fundamental analizar la base matemática que define cada modelo de proyección. En computación gráfica, las proyecciones tridimensionales se representan mediante matrices de transformación que modifican la manera en que los objetos del espacio 3D son proyectados sobre un plano bidimensional de visualización. Dependiendo de la configuración de dichas matrices, es posible obtener representaciones paralelas o perspectivas, alterando propiedades visuales como el paralelismo, la profundidad aparente, el escorzo y la convergencia de líneas.

En esta sección se describen las formulaciones matemáticas correspondientes a cada una de las proyecciones implementadas en el laboratorio: proyección ortogonal, proyección gabinete, perspectiva simétrica y perspectiva oblicua off-axis. Asimismo, se analizan los parámetros geométricos utilizados en las funciones `glOrtho` y `glFrustum`, así como las transformaciones matriciales adicionales aplicadas para construir los distintos modelos de visualización tridimensional.

a. Proyección Ortogonal

La proyección ortogonal descarta la coordenada Z. La matriz generada por `glOrtho(l, r, b, t, n, f)` es:

$$M_{\text{ortho}} = \begin{bmatrix} 2/(r-l) & 0 & 0 & -(r+l)/(r-l) \\ 0 & 2/(t-b) & 0 & -(t+b)/(t-b) \\ 0 & 0 & -2/(f-n) & -(f+n)/(f-n) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Propiedades: conservación del paralelismo, ausencia de punto de fuga, sin escorzo. Es la proyección estándar en CAD y planos de ingeniería.

b. Proyección Gabinete

La proyección gabinete es una proyección oblicua paralela. Se construye como una ortogonal base a la que se aplica una transformación de cizallamiento (shear) con ángulo de 30 grados y factor $L = 0.5$:

$$M_{\text{shear}} = \begin{bmatrix} 1 & 0 & \cos(30^\circ) \cdot 0.5 & 0 \\ 0 & 1 & \sin(30^\circ) \cdot 0.5 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (\text{column-major, OpenGL})$$

$$\cos(30^\circ) \cdot 0.5 = 0.8660 \cdot 0.5 = 0.4330$$

$$\sin(30^\circ) \cdot 0.5 = 0.5000 \cdot 0.5 = 0.2500$$

c. Proyección Perspectiva Simétrica

Se genera con `glFrustum` centrado (left=-right, bottom=-top). La matriz resultante tiene los términos de simetría en cero:

Parámetros: $\text{fovy}=40^\circ$, $\text{near}=2$, $\text{far}=20$
 $\text{half} = \tan(20^\circ) \cdot \text{near} = 0.364 \cdot 2 = 0.728$
`glFrustum(-0.728, 0.728, -0.728, 0.728, 2, 20)`

El eje del frustum coincide con -Z.
Un único punto de fuga central.

d. Proyección Perspectiva Oblicua

`glFrustum` asimétrico (off-axis). El frustum está descentrado respecto al eje Z del ojo:

$\text{offX} = 0.3$, $\text{offY} = 0.25$
`glFrustum(-0.728+0.3, 0.728+0.3,`
`-0.728+0.25, 0.728+0.25,`

2.0, 20.0)

=> glFrustum(-0.428, 1.028, -0.478, 0.978, 2, 20)

El punto de fuga se desplaza fuera del eje central.
Técnica usada en VR, displays estéreo y CAVE.

4. Arquitectura del Programa

La arquitectura del programa fue diseñada siguiendo un enfoque modular y orientado a eventos, aprovechando la estructura proporcionada por JOGL para la administración del ciclo de vida del contexto OpenGL y el proceso de renderizado gráfico. La aplicación se organiza mediante componentes especializados encargados de inicializar el entorno gráfico, gestionar la ventana de visualización, controlar los eventos del sistema y renderizar las diferentes proyecciones tridimensionales implementadas en el laboratorio.

El sistema utiliza el patrón `GLEventListener`, ampliamente empleado en aplicaciones desarrolladas con JOGL, permitiendo separar de manera estructurada las etapas de inicialización, renderizado, redimensionamiento y liberación de recursos gráficos. Asimismo, la arquitectura implementada divide la ventana principal en múltiples viewports independientes, posibilitando la representación simultánea de distintas configuraciones de proyección sobre un mismo objeto geométrico. Esta organización facilita el análisis comparativo entre proyecciones ortogonales y perspectivas, además de mejorar la claridad, mantenibilidad y escalabilidad del código fuente.

En las siguientes subsecciones se describe detalladamente la estructura general del programa, el flujo de ejecución de los principales métodos y la manera en que cada componente participa en el proceso de renderizado tridimensional utilizando OpenGL y Java.

a. Estructura general

El programa implementa el patrón `GLEventListener` de JOGL. La clase `Laboratorio02` implementa `GLEventListener` y `KeyListener`. El main inicializa el contexto OpenGL y crea el `JFrame` con el `GLCanvas`. Los cuatro cuadrantes se renderizan en el callback `display()` usando `glViewport()` para dividir la ventana.

```
main()
|-- System.setProperty(sun.java2d.noddraw, true)
|-- SwingUtilities.invokeLater()
|-- GLProfile.initSingleton()
|-- GLProfile.get(GL2)
|-- GLCapabilities(profile)
|-- GLCanvas(caps)
|-- JFrame -> pack() -> setVisible()

display(GLAutoDrawable)
|-- proyeccionOrtogonal (0, H/2, W/2, H/2)
|-- proyeccionGabinete (W/2,H/2, W/2, H/2)
|-- proyeccionSimetrica (0, 0, W/2, H/2)
|-- proyeccionOblicua (W/2,0, W/2, H/2)
|-- dibujarSeparadores()
```

b. Patrón de renderizado por cuadrante

Cada función de proyección sigue el mismo patrón: (1) `glViewport` para definir la región de pantalla, (2) `GL_PROJECTION` con `glLoadIdentity` y la función de proyección específica, (3) `GL_MODELVIEW` con la transformación de modelo, (4) `dibujarCubo()` y `dibujarTexto()`.

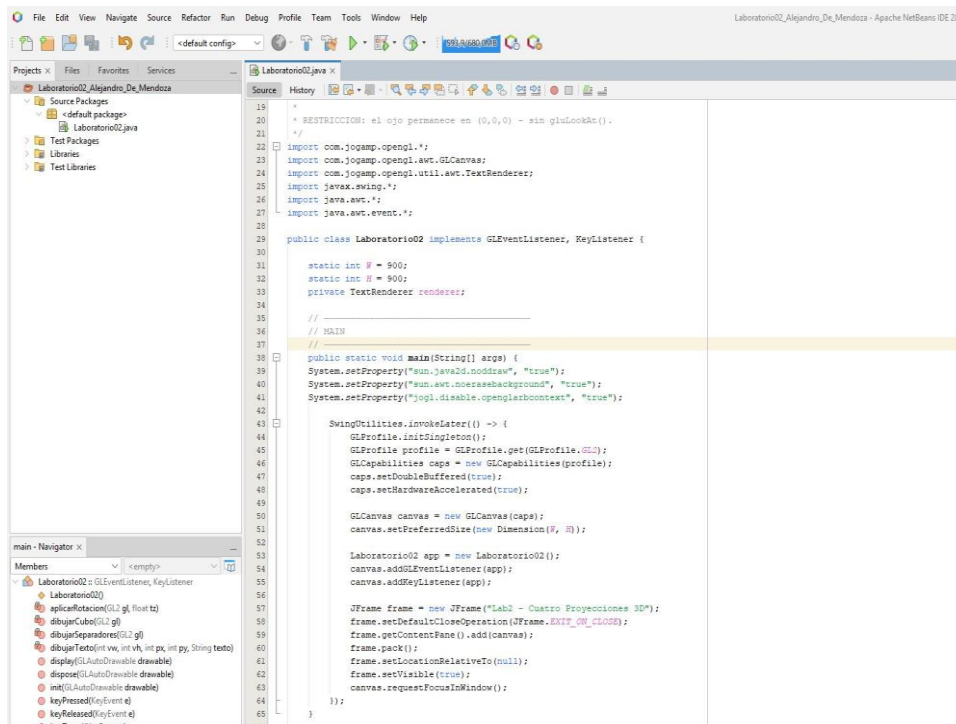


Figura 1. Imports JOGL, declaración de clase e inicio del método main con System.setProperty.

c. Método init() - Inicialización OpenGL

El método init() corresponde a la fase de inicialización del contexto OpenGL dentro del ciclo de vida del GLEventListener. Su función principal consiste en configurar los parámetros gráficos iniciales necesarios para el correcto funcionamiento de la aplicación, incluyendo el color de fondo, la activación del buffer de profundidad y la inicialización de los recursos utilizados durante el renderizado. Esta etapa resulta fundamental, ya que prepara el entorno gráfico antes de iniciar el proceso de representación de las diferentes proyecciones tridimensionales implementadas en el laboratorio.

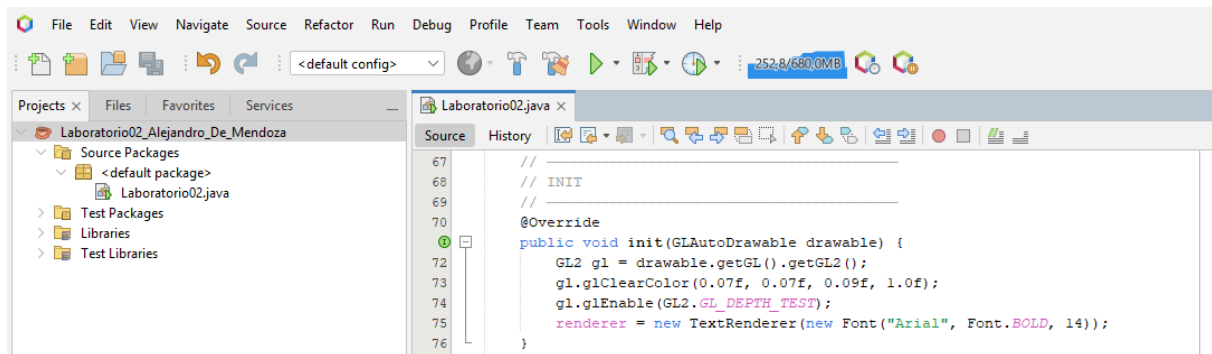


Figura 2. Método init(): configura color de fondo, activa depth test e inicializa TextRenderer.

d. Método reshape() - Redimensionado

El método reshape() se ejecuta cada vez que la ventana OpenGL cambia de tamaño, permitiendo actualizar dinámicamente las dimensiones del área de renderizado y mantener la correcta visualización de la escena gráfica. Su implementación resulta fundamental para adaptar los viewports y las proyecciones a las nuevas dimensiones de la ventana, evitando deformaciones visuales y garantizando que las cuatro representaciones tridimensionales conserven una distribución adecuada dentro de la interfaz gráfica.

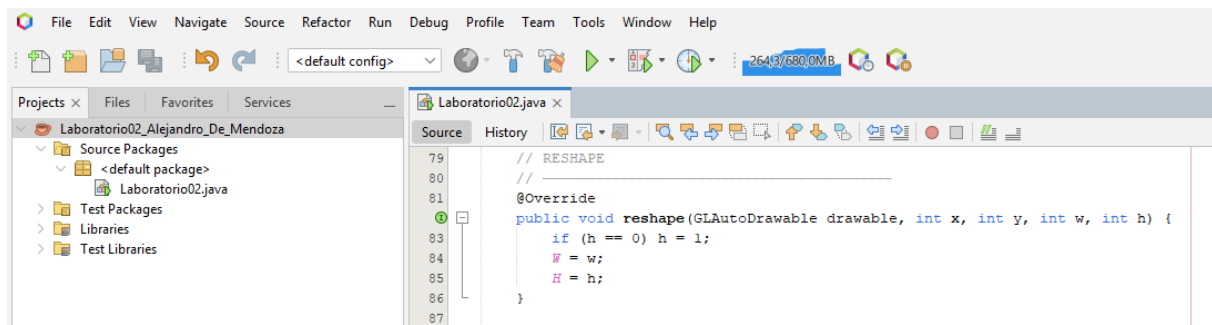


Figura 3. Método reshape(): actualiza dimensiones W y H cuando la ventana cambia de tamaño.

e. Método display() - Loop de renderizado

El método display() constituye el núcleo principal del proceso de renderizado de la aplicación, ya que se ejecuta continuamente durante el ciclo de vida del contexto OpenGL para actualizar y dibujar la escena gráfica en pantalla. En esta etapa se realiza la limpieza de los buffers, la configuración de cada viewport y la ejecución de las funciones encargadas de renderizar las distintas proyecciones tridimensionales del cubo. Gracias a este método, el sistema puede representar simultáneamente las cuatro configuraciones de proyección implementadas en el laboratorio, manteniendo una visualización organizada y coherente dentro de la ventana gráfica.

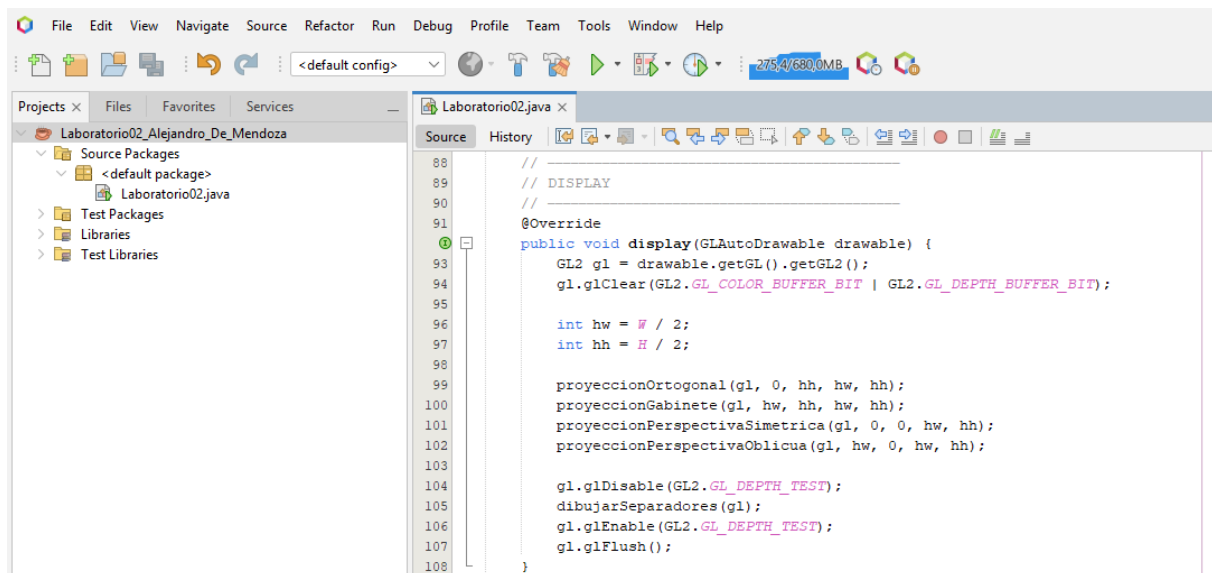


Figura 4. Método display(): limpia buffers y llama a las 4 funciones de proyección.

f. Función dibujarCubo() - Geometría del cubo

La función dibujarCubo() es la encargada de construir y renderizar el objeto tridimensional utilizado como elemento de prueba en las diferentes proyecciones implementadas. Su propósito principal consiste en definir la geometría del cubo mediante primitivas gráficas de OpenGL, especificando las coordenadas espaciales de sus vértices, las caras visibles y las aristas que conforman la estructura del objeto. Adicionalmente, se aplican diferentes colores sobre cada cara del cubo para facilitar la percepción visual de profundidad, orientación espacial y diferencias geométricas producidas por cada tipo de proyección tridimensional.

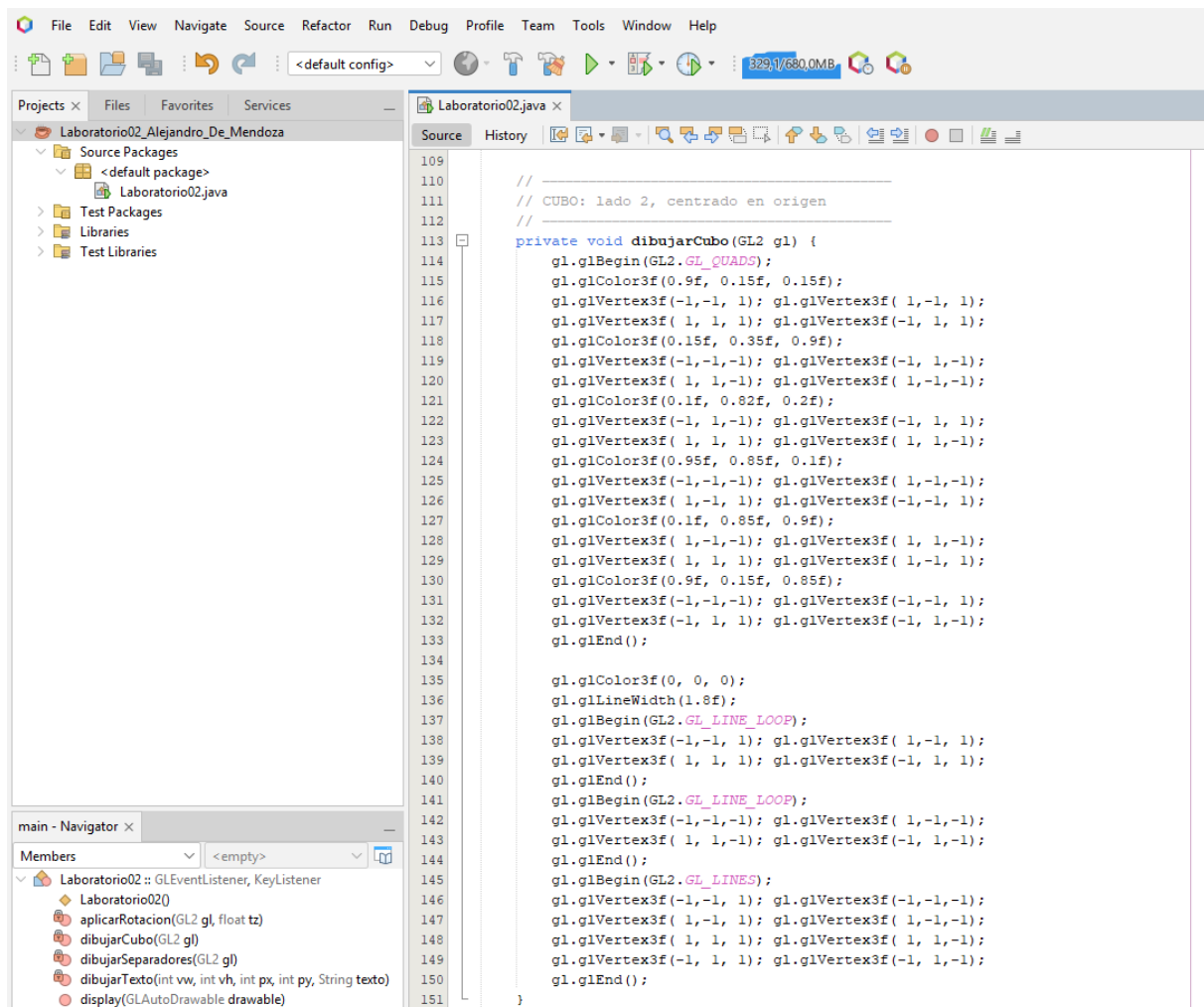


Figura 5. Cubo con 6 caras GL_QUADS coloreadas + aristas GL_LINE_LOOP y GL_LINES en negro.

g. Funciones aplicarRotacion() y dibujarTexto()

Las funciones aplicarRotacion() y dibujarTexto() cumplen un papel fundamental dentro del proceso de visualización de la escena tridimensional y de la interfaz gráfica del programa. La función aplicarRotacion() se encarga de posicionar y orientar el cubo dentro del espacio 3D mediante transformaciones de traslación y rotación, permitiendo visualizar simultáneamente varias de sus caras y facilitando el análisis comparativo entre las diferentes proyecciones implementadas. Por su parte, la función dibujarTexto() permite renderizar etiquetas informativas sobre cada cuadrante utilizando herramientas de texto proporcionadas por JOGL, mejorando la identificación visual de cada tipo de proyección y aportando mayor claridad a la presentación gráfica del laboratorio.

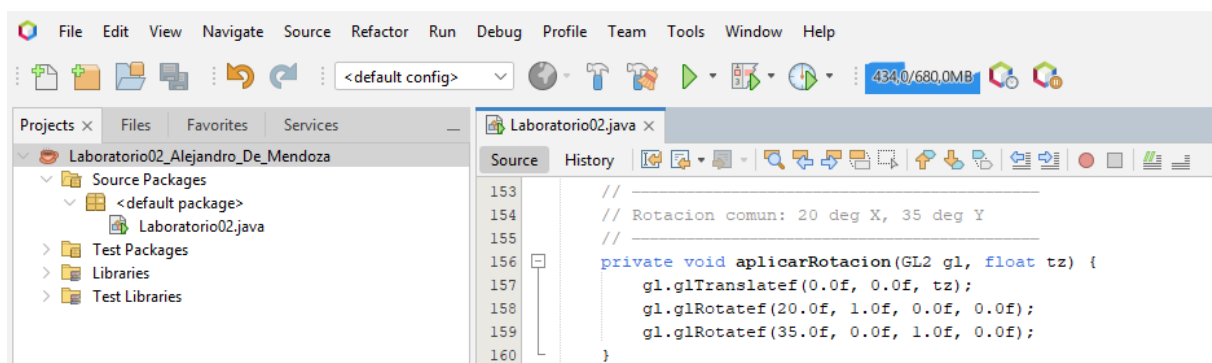


Figura 6. aplicarRotacion(): traslada en -Z y rota 20° en X, 35° en Y.

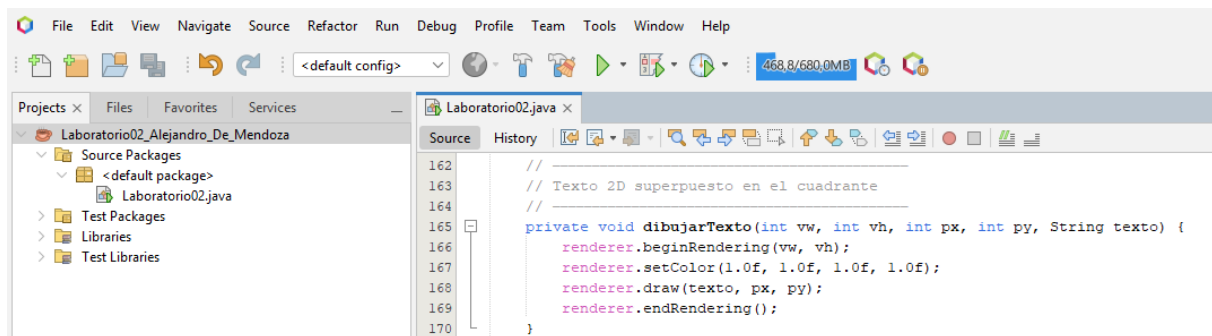


Figura 7. dibujarTexto(): renderiza etiqueta 2D usando TextRenderer de JOGL.

h. Proyección Ortogonal

La proyección ortogonal corresponde a una técnica de proyección paralela en la cual los rayos visuales permanecen paralelos entre sí y perpendiculares al plano de visualización. En este tipo de representación no existe convergencia de líneas ni efecto de perspectiva, por lo que los objetos conservan sus dimensiones independientemente de la profundidad. Esta característica permite mantener proporciones geométricas exactas, razón por la cual la proyección ortogonal es ampliamente utilizada en aplicaciones de ingeniería, diseño asistido por computador (CAD) y representación técnica de objetos tridimensionales.

En el desarrollo del laboratorio, la proyección ortogonal fue implementada mediante la función `glOrtho()`, configurando un volumen de visión rectangular y manteniendo el ojo fijo en el origen del sistema de coordenadas. Esta implementación permite observar claramente la ausencia de escorzo y punto de fuga, facilitando la comparación visual con las demás proyecciones perspectivas desarrolladas en el programa.

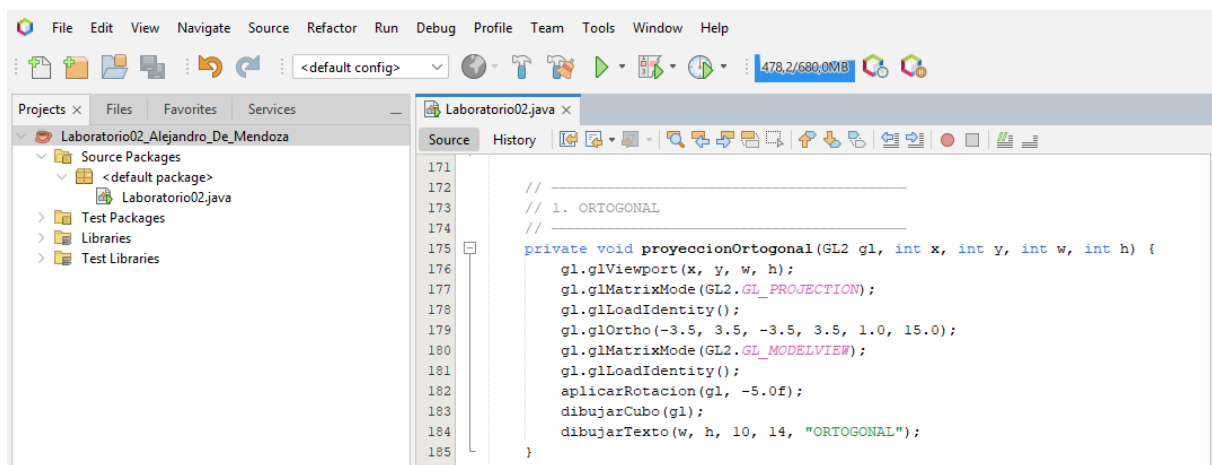


Figura 8. proyeccionOrtogonal(): `glOrtho(-3.5, 3.5, -3.5, 3.5, 1.0, 15.0)`. Sin punto de fuga.

i. Proyección Gabinete

La proyección gabinete corresponde a un tipo de proyección oblicua paralela utilizada para representar objetos tridimensionales conservando la escala real sobre el plano frontal mientras se incorpora una percepción parcial de profundidad. A diferencia de la proyección ortogonal, esta técnica aplica una transformación de cizallamiento sobre el eje de profundidad, permitiendo visualizar las dimensiones del objeto en el eje Z sin generar convergencia perspectiva. Debido a ello, las líneas paralelas permanecen paralelas, pero la profundidad se representa mediante una inclinación oblicua controlada matemáticamente.

En el desarrollo del laboratorio, la proyección gabinete fue implementada utilizando una proyección ortogonal base mediante `glOrtho()`, sobre la cual se aplicó manualmente una matriz de cizallamiento con un ángulo de 30 grados y un factor de reducción $L = 0.5$. Esta configuración permite obtener una representación visual más intuitiva de la profundidad del cubo, manteniendo al mismo tiempo proporciones geométricas relativamente estables. La implementación realizada facilita el análisis

comparativo entre las proyecciones paralelas puras y las proyecciones perspectivas desarrolladas en los demás cuadrantes del sistema.

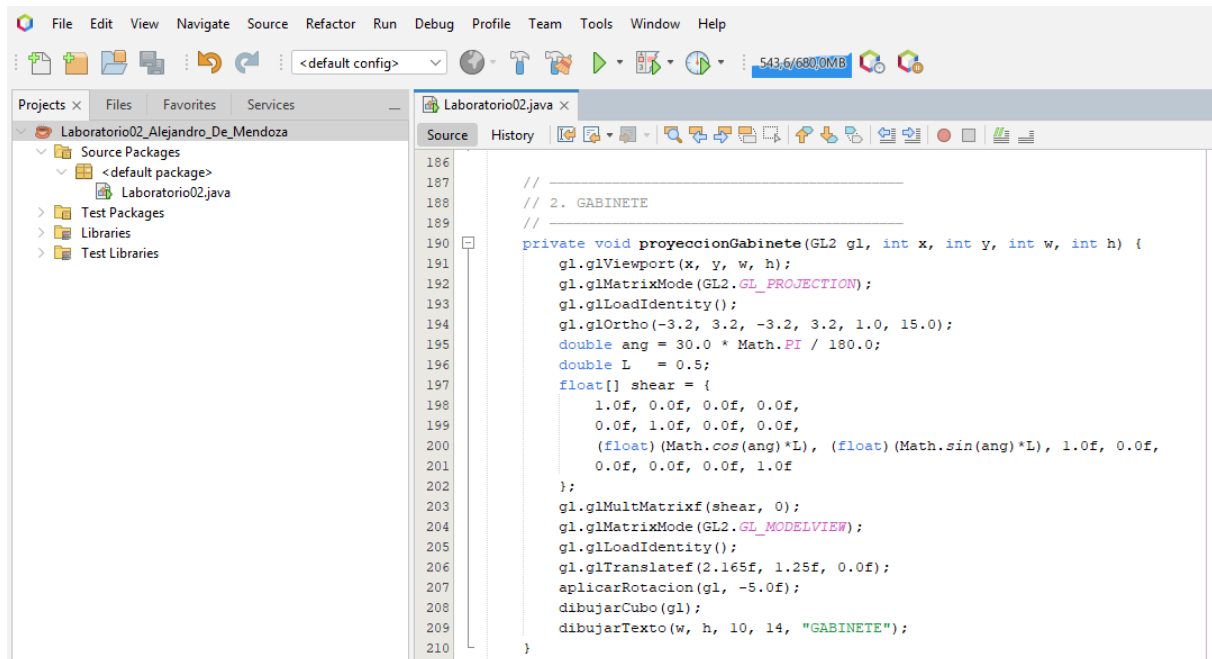


Figura 9. proyeccionGabinete(): glOrtho + matriz shear 30° L=0.5 aplicada con glMultMatrixf.

j. Proyección Perspectiva Simétrica

La proyección perspectiva simétrica corresponde a un modelo de proyección cónica en el cual los rayos visuales convergen en un único punto de observación ubicado en el origen del sistema de coordenadas. A diferencia de las proyecciones paralelas, este tipo de representación genera un efecto de profundidad más realista, provocando que los objetos lejanos aparenten un menor tamaño debido al fenómeno de escorzo. Como resultado, las líneas paralelas tienden visualmente a converger hacia un punto de fuga central, reproduciendo de manera más precisa la percepción visual humana.

En el desarrollo del laboratorio, la proyección perspectiva simétrica fue implementada mediante la función glFrustum() utilizando un frustum centrado respecto al eje Z, donde los límites izquierdo y derecho, así como superior e inferior, mantienen una distribución simétrica. Esta configuración genera un volumen de visión balanceado y alineado con el eje principal de observación, permitiendo visualizar el cubo tridimensional con un punto de fuga central claramente definido. La implementación facilita el análisis de los efectos de profundidad y perspectiva en comparación con las proyecciones paralelas desarrolladas en los demás cuadrantes del programa.

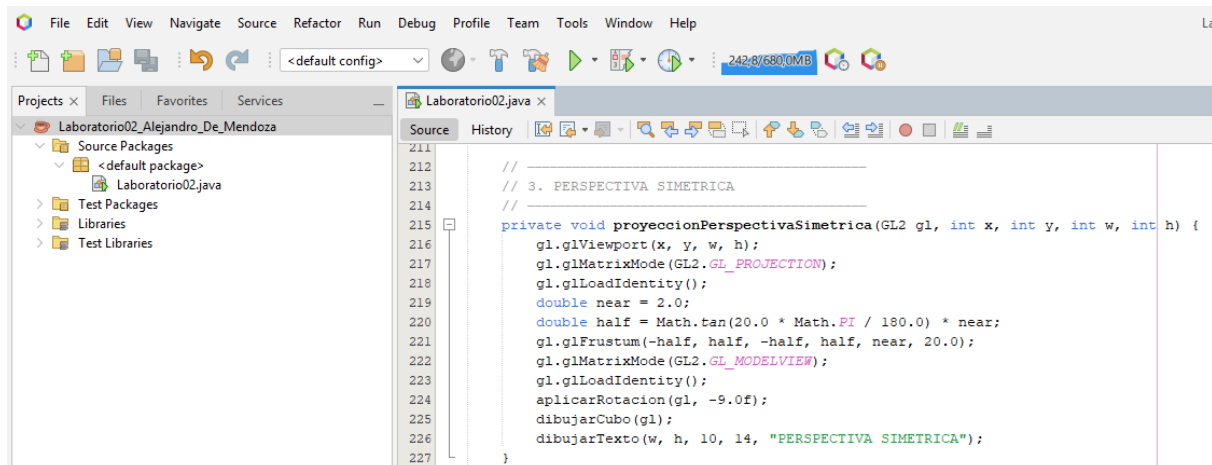


Figura 10. proyeccionPerspectivaSimetrica(): glFrustum centrado, fovy=40°, near=2, far=20.

k. Proyección Perspectiva Oblicua

La proyección perspectiva oblicua, también conocida como proyección off-axis, corresponde a una variante de la proyección perspectiva en la cual el frustum de visión se encuentra descentrado respecto al eje principal de observación. A diferencia de la perspectiva simétrica, donde el volumen de visión se alinea perfectamente con el eje Z, la proyección oblicua desplaza lateral y verticalmente el frustum, generando un punto de fuga fuera del centro de la escena. Este tipo de proyección es ampliamente utilizado en sistemas de realidad virtual, visualización estéreo, simuladores inmersivos y entornos CAVE, donde resulta necesario adaptar la perspectiva según la posición física del observador.

En el desarrollo del laboratorio, la proyección perspectiva oblicua fue implementada mediante la función `glFrustum()` utilizando desplazamientos horizontales y verticales definidos por los parámetros `offX` y `offY`. Esta configuración produce una representación visual asimétrica del cubo tridimensional, permitiendo observar cómo el descentramiento del frustum modifica la percepción espacial y la ubicación aparente del punto de fuga dentro de la escena. La implementación facilita el análisis comparativo entre perspectivas simétricas y off-axis, evidenciando el impacto geométrico y visual de las transformaciones aplicadas sobre el volumen de visión.

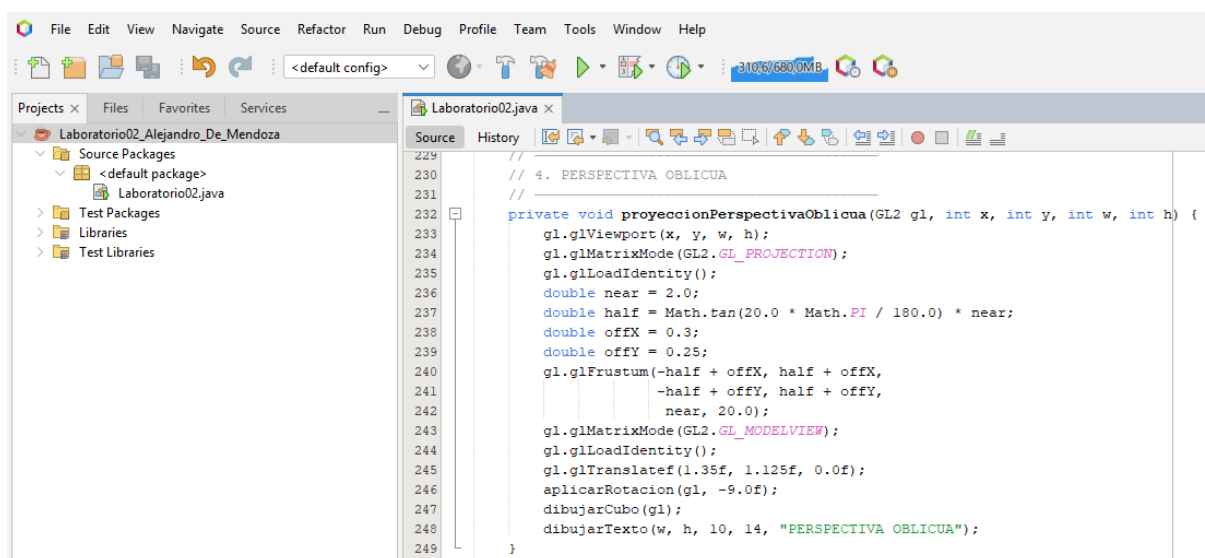


Figura 11. `proyeccionPerspectivaOblicua()`: `glFrustum` off-axis con `offX=0.3`, `offY=0.25`.

l. Separadores y Dispose

La implementación de los separadores y del método `dispose()` cumple funciones complementarias dentro de la estructura gráfica y administrativa de la aplicación. Por un lado, los separadores visuales permiten dividir claramente la ventana OpenGL en cuatro cuadrantes independientes, facilitando la organización de las distintas proyecciones tridimensionales y mejorando la interpretación visual de los resultados obtenidos. Estas líneas de división se renderizan como elementos bidimensionales superpuestos sobre la escena principal, proporcionando una distribución más clara y estructurada de la interfaz gráfica.

Por otro lado, el método `dispose()` se encarga de liberar correctamente los recursos gráficos utilizados por la aplicación cuando el contexto OpenGL es destruido o el programa finaliza su ejecución. Esta etapa resulta importante para evitar fugas de memoria y garantizar una gestión adecuada de los objetos asociados al renderizado, especialmente aquellos relacionados con componentes auxiliares como el sistema de renderizado de texto implementado mediante `TextRenderer` en JOGL.

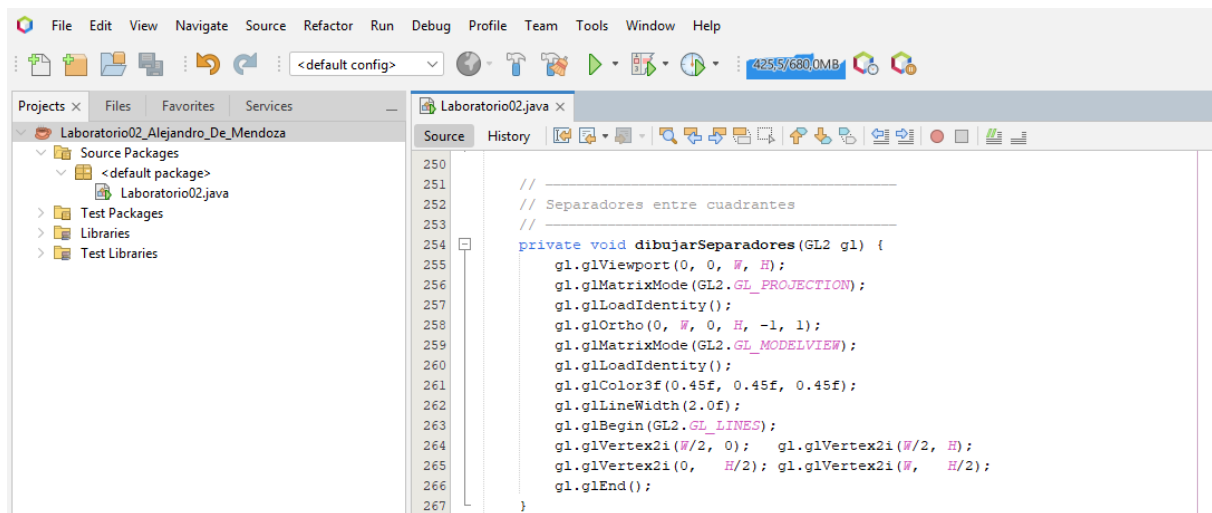


Figura 12. dibujarSeparadores(): líneas grises en overlay 2D entre los cuadrantes.

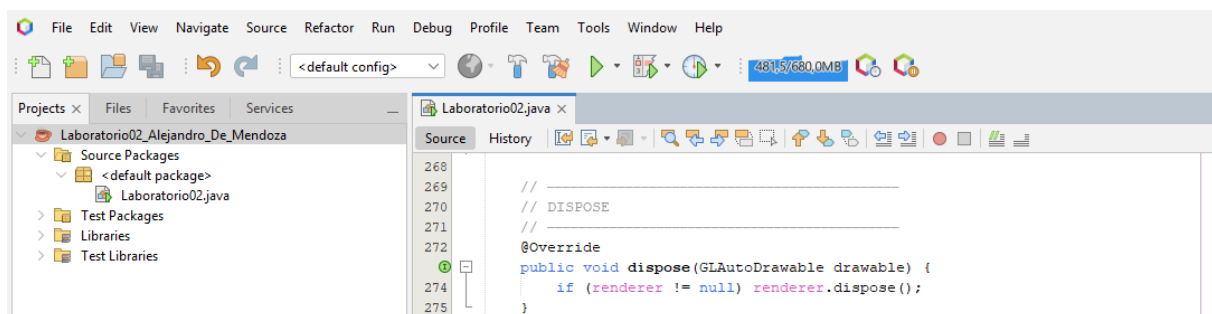


Figura 13. dispose(): libera el TextRenderer al cerrar el contexto OpenGL.

m. Teclado

La gestión de eventos de teclado permite incorporar interacción básica entre el usuario y la aplicación gráfica, facilitando el control del programa durante su ejecución. En el contexto del laboratorio, se implementó el manejo de eventos mediante la interfaz KeyListener de Java, permitiendo detectar pulsaciones de teclas y ejecutar acciones específicas asociadas al sistema de visualización.

Particularmente, se configuró la tecla ESC como mecanismo de cierre de la aplicación, permitiendo finalizar la ejecución del programa de manera segura y controlada. Aunque las funciones keyReleased() y keyTyped() no contienen lógica adicional en esta implementación, su inclusión mantiene la estructura completa del sistema de eventos requerida por la interfaz, dejando preparada la arquitectura para futuras ampliaciones relacionadas con interacción gráfica, navegación de cámara o manipulación dinámica de objetos tridimensionales.

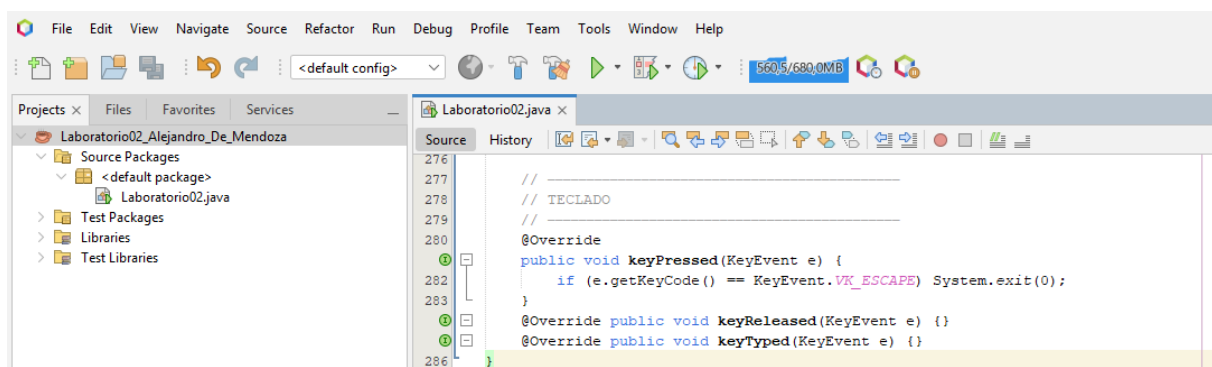


Figura 14. keyPressed(): ESC cierra la aplicación. keyReleased y keyTyped vacíos.

5. Configuración del Entorno en NetBeans

La correcta configuración del entorno de desarrollo constituye un aspecto fundamental para garantizar el funcionamiento adecuado de aplicaciones gráficas basadas en OpenGL y JOGL. Debido a que este tipo de aplicaciones requiere la integración de bibliotecas gráficas, dependencias nativas y compatibilidad con versiones específicas de Java, resulta necesario establecer cuidadosamente la estructura del proyecto y la configuración del entorno de ejecución dentro del IDE utilizado.

En el desarrollo del presente laboratorio se empleó Apache NetBeans IDE 28 junto con Java 8 (Zulu 8.0.492+9) y las bibliotecas JOGL correspondientes, configurando manualmente los archivos JAR y los parámetros de ejecución necesarios para la correcta inicialización del contexto OpenGL. En las siguientes subsecciones se describen detalladamente las dependencias utilizadas, la configuración de la plataforma Java y los principales ajustes realizados en el proyecto para garantizar la ejecución estable de la aplicación gráfica tridimensional.

a. Librerías del proyecto

Las librerías del proyecto constituyen los componentes fundamentales que permiten establecer la comunicación entre la aplicación desarrollada en Java y el sistema gráfico OpenGL. En el caso de JOGL, estas dependencias proporcionan tanto las interfaces de programación necesarias para acceder a las funciones de OpenGL como las bibliotecas nativas encargadas de interactuar directamente con el hardware gráfico y el sistema operativo.

Para el desarrollo del laboratorio se incorporaron manualmente los archivos JAR correspondientes a JOGL y GlueGen dentro del classpath del proyecto en Apache NetBeans IDE 28. Esta configuración garantiza la correcta inicialización del contexto OpenGL, el acceso a las funciones gráficas de bajo nivel y la compatibilidad con el entorno de ejecución utilizado. Asimismo, las bibliotecas nativas permiten habilitar la aceleración por hardware necesaria para el renderizado eficiente de las diferentes proyecciones tridimensionales implementadas en la aplicación.

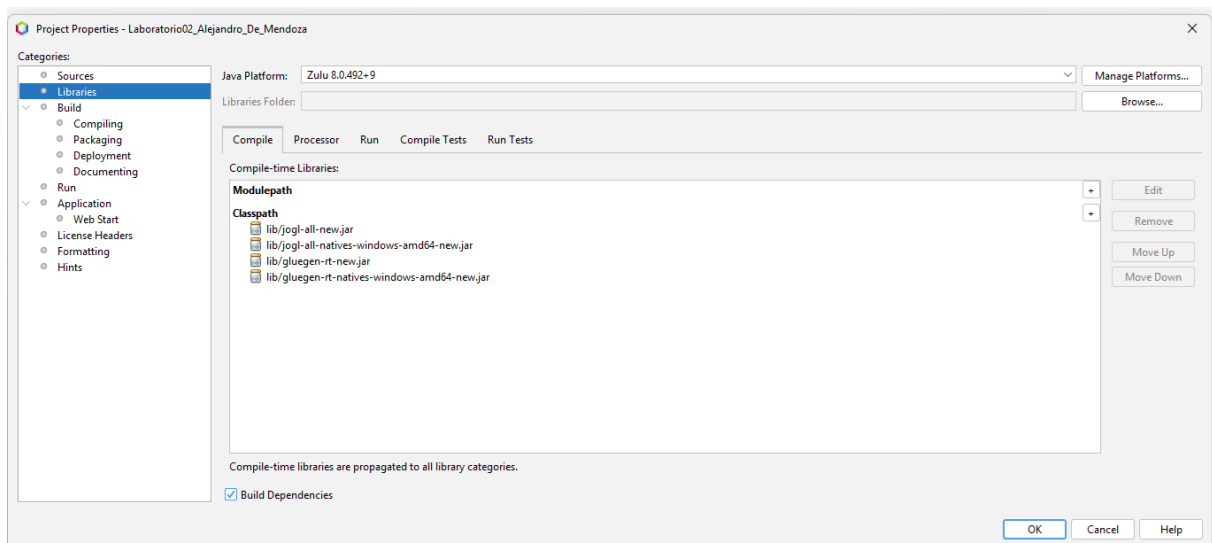


Figura 15. Project Properties: 4 JARs JOGL en Classpath, plataforma Zulu 8.0.492+9.

b. Java Platform Manager

El Java Platform Manager de Apache NetBeans IDE permite administrar las diferentes versiones del entorno de ejecución Java utilizadas por los proyectos desarrollados dentro del IDE. Esta herramienta resulta especialmente importante en aplicaciones basadas en JOGL, debido a que la compatibilidad entre OpenGL, las bibliotecas gráficas y la versión del JDK puede afectar directamente la correcta inicialización del contexto gráfico y la ejecución del programa.

En el desarrollo del laboratorio se configuró la plataforma Zulu JDK 8 (Java SE 8) como entorno principal de compilación y ejecución, garantizando compatibilidad con las bibliotecas JOGL utilizadas en el proyecto. La correcta configuración de esta plataforma permitió establecer el entorno necesario para el manejo de dependencias gráficas, ejecución de aplicaciones OpenGL y funcionamiento estable del sistema de renderizado tridimensional implementado en Java.

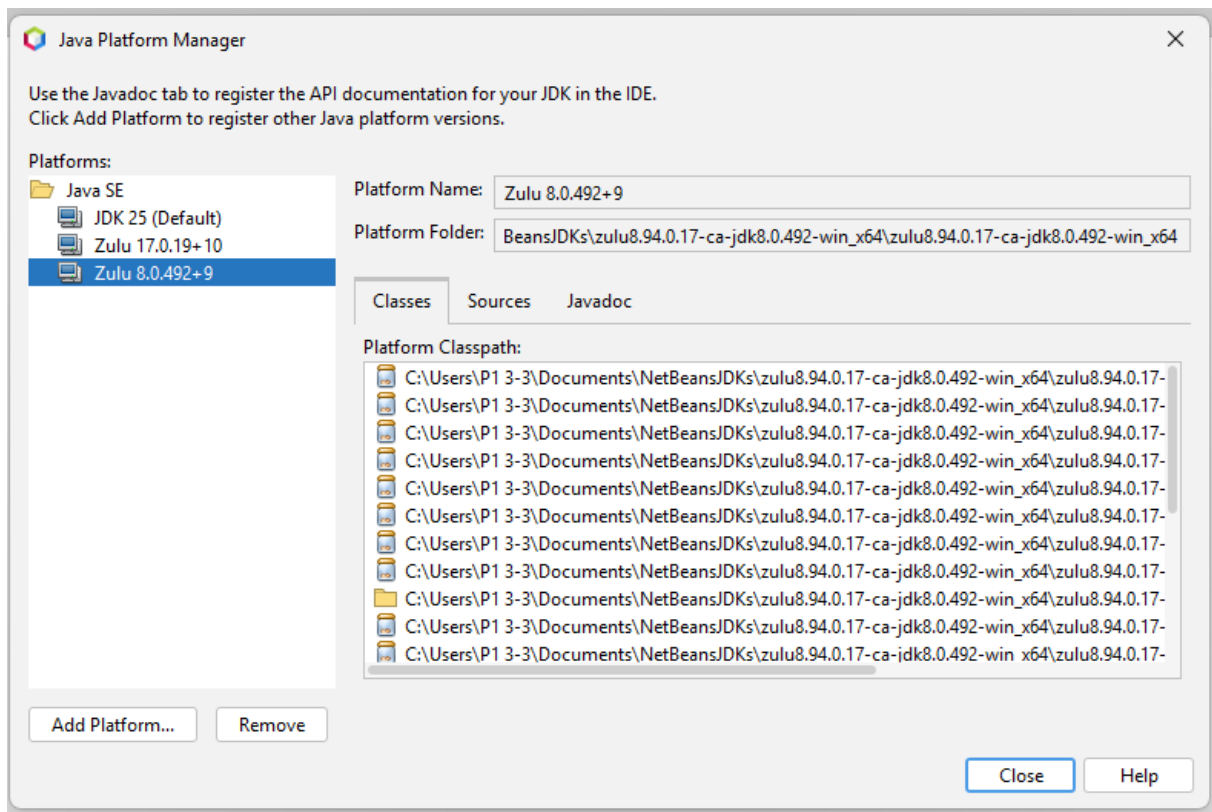


Figura 16. Java Platform Manager con Zulu 8 (Java 8) registrado y activo para el proyecto.

c. Archivo project.properties

El archivo `project.properties` corresponde al archivo de configuración interna utilizado por Apache NetBeans IDE para almacenar los parámetros de compilación, ejecución y administración de dependencias del proyecto Java. Este archivo desempeña un papel fundamental dentro de la estructura del entorno de desarrollo, ya que permite definir las bibliotecas externas utilizadas, las rutas de ejecución, la plataforma Java seleccionada y los argumentos necesarios para el correcto funcionamiento de la aplicación.

En el desarrollo del laboratorio, el archivo `project.properties` fue configurado manualmente para incluir las dependencias correspondientes a JOGL y GlueGen, así como los parámetros JVM requeridos para evitar conflictos entre Java2D y el contexto OpenGL en sistemas Windows. Esta configuración garantiza la correcta carga de las bibliotecas nativas, la compatibilidad con Java 8 y la ejecución estable del sistema de renderizado tridimensional implementado en el proyecto.



```
1 annotation.processing.enabled=true
2 annotation.processing.enabled.in.editor=false
3 annotation.processing.processors.list=
4 annotation.processing.run.all.processors=true
5 annotation.processing.source.output=${build.generated.sources.dir}/ap-source-output
6 application.title=Laboratorio02_Alejandro_De_Mendoza
7 application.vendor=Alejo
8 build.classes.dir=${build.dir}/classes
9 build.classes.excludes=**/*.java,**/*.form
10 build.dir=build
11 build.generated.dir=${build.dir}/generated
12 build.generated.sources.dir=${build.dir}/generated-sources
13 build.sysclasspath=ignore
14 build.test.classes.dir=${build.dir}/test/classes
15 build.test.results.dir=${build.dir}/test/results
16 debug.classpath=\
17     ${run.classpath}
18 debug.modulepath=\
19     ${run.modulepath}
20 debug.test.classpath=\
21     ${run.test.classpath}
22 debug.test.modulepath=\
23     ${run.test.modulepath}
24 dist.archive.excludes=
25 dist.dir=dist
26 dist.jar=${dist.dir}/Laboratorio02_Alejandro_De_Mendoza.jar
27 dist.javadoc.dir=${dist.dir}/javadoc
28 dist.jlink.dir=${dist.dir}/jlink
29 dist.jlink.output=${dist.jlink.dir}/Laboratorio02_Alejandro_De_Mendoza
30 endorsed.classpath=
31 excludes=
32 file.reference.jogl-all-lib/jogl-all-new.jar
33 file.reference.jogl-natives-lib/jogl-all-natives-windows-amd64-new.jar
34 file.reference.gluegen-rt-lib/gluegen-rt-new.jar
35 file.reference.gluegen-natives-lib/gluegen-rt-natives-windows-amd64-new.jar
36 includes=**
37 jar.compress=false
38 javac.classpath=\
39     ${file.reference.jogl-all}:\
40     ${file.reference.jogl-natives}:\
41     ${file.reference.gluegen-rt}:\
42     ${file.reference.gluegen-natives}
43 javac.compilerargs=
44 javac.deprecation=false
45 javac.external.vm=true
46 javac.modulepath=
47 javac.processorpath=\
48     ${javac.classpath}
49 javac.source=1.8
50 javac.target=1.8
51 javac.test.classpath=\
52     ${javac.classpath}:\
53     ${build.classes.dir}
54 javac.test.modulepath=\
55     ${javac.modulepath}
56 javac.test.processorpath=\
57     ${javac.test.classpath}
58 javadoc.additionalparam=
59 javadoc.author=false
60 javadoc.encoding=${source.encoding}
61 javadoc.html5=false
62 javadoc.noindex=false
63 javadoc.nonavbar=false
64 javadoc.notree=false
65 javadoc.private=false
66 javadoc.splitindex=true
67 javadoc.use=true
68 javadoc.version=false
69 javadoc.windowtitle=
70 jlink.additionalmodules=
71 jlink.additionalparam=
72 jlink.launcher=true
73 jlink.launcher.name=Laboratorio02_Alejandro_De_Mendoza
74 main.class=Laboratorio02
75 manifest.file=manifest.mf
76 meta.inf.dir=${src.dir}/META-INF
77 mkdist.disabled=false
78 platform.active=Zulu_8.0.492_9
79 run.classpath=\
80     ${javac.classpath}:\
81     ${build.classes.dir}
82 run.jvmargs=-Dsun.java2d.noddraw=true -Dsun.awt.noerasebackground=true -Djava.library.path=lib
83 run.modulepath=\
84     ${javac.modulepath}
85 run.test.classpath=\
86     ${javac.test.classpath}:\
87     ${build.test.classes.dir}
88 run.test.modulepath=\
89     ${javac.test.modulepath}
90 source.encoding=UTF-8
91 src.dir=src
92 test.src.dir=test
```

Figura 17. project.properties con los 4 JARs referenciados individualmente y run.jvmargs configurado.

6. Captura de Pantalla del Programa en Ejecución

La siguiente sección presenta la evidencia visual del funcionamiento final de la aplicación desarrollada durante el laboratorio. La captura de pantalla permite observar la correcta ejecución del programa dentro del entorno Apache NetBeans IDE 28, así como la representación simultánea de las cuatro técnicas de proyección tridimensional implementadas sobre el mismo objeto geométrico.

La ventana OpenGL fue dividida en cuatro cuadrantes independientes mediante el uso de viewports, permitiendo visualizar de manera comparativa las diferencias geométricas y perceptuales entre la

proyección ortogonal, la proyección gabinete, la perspectiva simétrica y la perspectiva oblicua off-axis. Asimismo, la captura evidencia el correcto funcionamiento de las transformaciones de rotación, el manejo del buffer de profundidad y la organización visual de la interfaz gráfica desarrollada utilizando JOGL y OpenGL en Java.

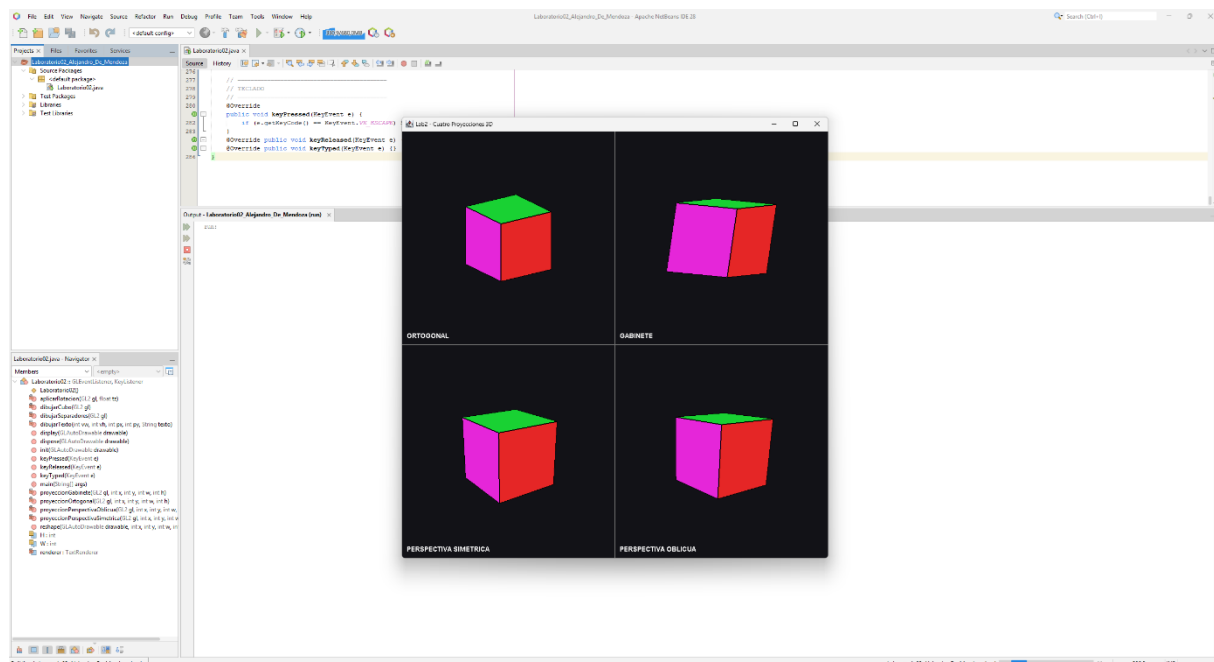


Figura 18. Programa corriendo en NetBeans IDE 28 con Java 8. Ventana 900x900 px con las 4 proyecciones.

La figura muestra el programa en ejecución dentro de Apache NetBeans IDE 28. La ventana OpenGL de 900x900 px se divide en cuatro cuadrantes iguales (450x450 px). El mismo cubo geométrico se renderiza con cuatro configuraciones de proyección independientes. La rotación de 20 grados en X y 35 grados en Y permite visualizar tres caras del cubo en cada cuadrante, facilitando la comparación del efecto de cada tipo de proyección.

7. Análisis Comparativo de las Proyecciones

El análisis comparativo de las proyecciones constituye una etapa fundamental dentro del laboratorio, ya que permite identificar y comprender las diferencias matemáticas, geométricas y visuales existentes entre los distintos modelos de representación tridimensional implementados en OpenGL. Aunque todas las proyecciones utilizan el mismo objeto geométrico y el mismo sistema de coordenadas, cada configuración de proyección modifica de manera significativa la percepción espacial, la profundidad aparente, el paralelismo de las líneas y la ubicación de los puntos de fuga dentro de la escena.

Mediante la comparación entre las proyecciones ortogonal, gabinete, perspectiva simétrica y perspectiva oblicua off-axis, es posible analizar cómo las matrices de transformación y los parámetros del volumen de visión influyen directamente en la representación final del objeto tridimensional. Asimismo, este análisis permite comprender las ventajas, limitaciones y aplicaciones prácticas de cada técnica de proyección dentro de áreas como ingeniería, diseño asistido por computador, simulación, videojuegos y sistemas de realidad virtual.

a. Tabla comparativa

La siguiente tabla presenta una comparación técnica y visual entre las diferentes proyecciones tridimensionales implementadas en el laboratorio. El análisis permite identificar las principales características geométricas de cada modelo de representación, así como las diferencias relacionadas con el comportamiento de los rayos visuales, la presencia de puntos de fuga, la percepción de profundidad y el efecto de perspectiva sobre el objeto renderizado.

Esta comparación facilita la comprensión de cómo las distintas configuraciones de las matrices de proyección influyen directamente en la representación final de la escena tridimensional, permitiendo establecer las ventajas y aplicaciones particulares de cada técnica dentro del ámbito de la informática gráfica y la visualización computacional.

Característica	Paralela (Ortogonal / Gabinete)	Perspectiva (Simétrica / Oblicua)
Función OpenGL	glOrtho + [shear opcional]	glFrustum
Punto de vista	En el infinito	En el origen (0,0,0)
Rayos visuales	Paralelos entre si	Convergen en el ojo
Punto de fuga	Ninguno	Uno central o descentrado
Escorzo	Ninguno	Si (lejano = más pequeño)
Conserva paralelismo	Si	No
Profundidad aparente	Igual a la real / mitad	Disminuye con la distancia
Uso típico	CAD, planos, isométrico	Videojuegos, simulación, VR

b. Diferencias observadas

- Ortogonal vs Gabinete: en la ortogonal las aristas paralelas en Z se superponen. En el gabinete el cizallamiento expone la profundidad con ángulo de 30 grados sin convergencia perspectiva.
- Ortogonal vs Perspectiva simétrica: la cara frontal es un cuadrado perfecto en la ortogonal. En la perspectiva simétrica hay escorzo claro: la cara frontal parece mayor que la trasera.
- Perspectiva simétrica vs oblicua: en la simétrica el cubo está centrado respecto al eje de visión. En la oblicua el cubo aparece desplazado visualmente, reflejando el descentramiento del frustum off-axis.

8. Funcionamiento y Fallos

La evaluación del funcionamiento del programa y el análisis de sus limitaciones técnicas permiten verificar el cumplimiento de los objetivos planteados en el laboratorio, así como identificar aspectos susceptibles de mejora dentro de la implementación desarrollada. Esta etapa resulta fundamental para validar el comportamiento de las diferentes proyecciones tridimensionales implementadas y analizar cómo las decisiones de diseño adoptadas afectan el rendimiento, la precisión geométrica y la calidad visual de la aplicación.

En las siguientes subsecciones se describen tanto los elementos que funcionaron correctamente durante la ejecución del sistema como las principales limitaciones técnicas observadas durante el desarrollo del laboratorio. Asimismo, se presentan posibles mejoras relacionadas con modernización del pipeline gráfico, optimización del renderizado y adaptación a arquitecturas gráficas más avanzadas basadas en OpenGL moderno y shaders programables.

a. Funcionamiento general

El programa funciona correctamente. Todos los requisitos del laboratorio han sido implementados y verificados:

1. La ventana OpenGL se divide en cuatro cuadrantes iguales mediante glViewport().
2. El ojo permanece en (0,0,0) en todos los cuadrantes. No se usa gluLookAt() en ningún lugar del código.
3. La proyección ortogonal usa glOrtho y muestra correctamente la ausencia de punto de fuga.
4. La proyección gabinete aplica cizallamiento matricial con convención estándar (30°, L=0.5).
5. La perspectiva simétrica usa glFrustum centrado con fovy=40 grados.
6. La perspectiva oblicua usa glFrustum off-axis con desplazamiento (0.3, 0.25).
7. Cada cuadrante muestra su etiqueta identificadora en overlay 2D con TextRenderer.
8. El programa responde al redimensionado de ventana mediante el callback reshape().

9. La tecla ESC cierra el programa limpiamente.

b. Limitaciones técnicas

A continuación, la lista de limitaciones encontradas:

- Limitación 1 - Fixed-Function Pipeline: El programa usa OpenGL 1.x/2.x (glBegin/glEnd, glMatrixMode), marcado como deprecated desde OpenGL 3.0. Una implementación moderna usaría shaders GLSL, VAOs y VBOs.
- Limitación 2 - Sin iluminación: El renderizado usa color plano sin cálculo de normales ni modelo de iluminación (GL_LIGHTING). La ausencia de sombreado limita la percepción de volumen.
- Limitación 3 - Shear en GL_PROJECTION: La matriz de cizallamiento gabinete se aplica sobre GL_PROJECTION. La práctica correcta sería aplicarla en GL_MODELVIEW, pero esto requeriría modificar la restricción del ojo en el origen.
- Limitación 4 - Offsets manuales: Las proyecciones gabinete y oblicua requieren una glTranslatef compensatoria determinada empíricamente. Una solución robusta calcularía estos valores automáticamente a partir de los parámetros de shear y off-axis.
- Limitación 5 - Aspect ratio fijo: El frustum usa aspect ratio 1:1. Si el usuario redimensiona a proporciones no cuadradas, el cubo se deformará. La corrección sería calcular $\text{half_x} = \text{half} * (\text{w/h})$ en cada reshape.

9. Entorno de Desarrollo

El entorno de desarrollo utilizado para la implementación del laboratorio fue configurado con el objetivo de garantizar compatibilidad, estabilidad y correcto funcionamiento del contexto gráfico OpenGL dentro del lenguaje Java. Debido a que las aplicaciones basadas en JOGL requieren integración entre bibliotecas gráficas, dependencias nativas y aceleración por hardware, fue necesario establecer cuidadosamente tanto la plataforma de desarrollo como los parámetros de ejecución del proyecto.

Para el desarrollo de la aplicación se empleó Apache NetBeans IDE 28 junto con Java 8 (Zulu 8.0.492+9) y las bibliotecas JOGL 2.4, configurando manualmente las dependencias y propiedades necesarias para la correcta inicialización del sistema gráfico. Asimismo, se utilizaron parámetros específicos de la máquina virtual Java para evitar conflictos entre Java2D y OpenGL en sistemas Windows, garantizando así un entorno de renderizado estable y compatible con el pipeline gráfico utilizado durante el laboratorio.

Sistema operativo	Windows 10/11 (64-bit)
IDE	Apache NetBeans IDE 28
Java Platform	Zulu 8.0.492+9 (Java SE 8)
Librería grafica	JOGL 2.4 (jogl-all + gluegen-rt)
OpenGL	OpenGL 2.1 / fixed-function pipeline
Render backend	WindowsWGL (hardware accelerated)

a. Dependencias JAR

Las dependencias JAR incorporadas en el proyecto corresponden a las bibliotecas necesarias para habilitar la integración entre Java y OpenGL mediante JOGL. Estos archivos proporcionan tanto las interfaces de programación utilizadas por la aplicación como los componentes nativos requeridos para establecer comunicación directa con el sistema gráfico y el hardware de renderizado.

En el desarrollo del laboratorio se utilizaron bibliotecas correspondientes a JOGL y GlueGen, incluyendo tanto los archivos principales de ejecución como sus respectivas dependencias nativas para

arquitectura Windows de 64 bits. La correcta incorporación de estos archivos dentro del classpath del proyecto permitió inicializar el contexto OpenGL, ejecutar las funciones gráficas del pipeline clásico y garantizar el funcionamiento estable de las diferentes proyecciones tridimensionales implementadas en la aplicación.

```
lib/jogl-all-new.jar          -- API JOGL principal
lib/jogl-all-natives-windows-amd64-new.jar -- DLLs nativos JOGL
lib/gluegen-rt-new.jar        -- Runtime GlueGen
lib/gluegen-rt-natives-windows-amd64-new.jar -- DLLs nativos GlueGen
```

b. Configuración JVM

La configuración de la JVM (Java Virtual Machine) desempeña un papel fundamental en la ejecución de aplicaciones gráficas desarrolladas con JOGL y OpenGL, ya que permite definir parámetros específicos relacionados con la administración de bibliotecas nativas, aceleración gráfica y compatibilidad entre Java2D y el contexto de renderizado OpenGL. Una configuración inadecuada puede generar conflictos visuales, problemas de inicialización del contexto gráfico o fallos durante la ejecución de la aplicación.

En el desarrollo del laboratorio se configuraron argumentos específicos dentro de la máquina virtual Java para optimizar la estabilidad del entorno gráfico en sistemas Windows. Particularmente, se deshabilitaron ciertos mecanismos de aceleración de Java2D y se estableció manualmente la ruta de las bibliotecas nativas utilizadas por JOGL y GlueGen. Esta configuración permitió garantizar la correcta inicialización del contexto OpenGL, el acceso a las DLLs nativas y el funcionamiento estable del sistema de renderizado tridimensional implementado en el proyecto.

```
run.jvmargs=-Dsun.java2d.noddraw=true
-Dsun.awt.noerasebackground=true
-Djava.library.path=lib
```

Razón: JOGL 2.x requiere deshabilitar DirectDraw de Java2D para evitar conflictos con el contexto WGL de OpenGL en Windows.

Y con este último punto se da por finalizado el desarrollo de este laboratorio,

CONCLUSIONES DE LA ACTIVIDAD

El desarrollo del presente laboratorio permitió cumplir satisfactoriamente los objetivos planteados en la actividad, logrando la implementación de un sistema de visualización de proyecciones tridimensionales mediante OpenGL y la biblioteca JOGL en lenguaje Java. A través de esta práctica se aplicaron de manera efectiva los fundamentos de la informática gráfica y la visualización computacional, especialmente aquellos relacionados con el pipeline gráfico, las transformaciones geométricas y las técnicas de proyección utilizadas para representar objetos tridimensionales sobre un plano bidimensional.

Durante el desarrollo del proyecto se utilizaron diferentes funciones y estructuras del pipeline clásico de OpenGL, entre ellas `glOrtho` para la implementación de proyecciones paralelas y `glFrustum` para la construcción de proyecciones perspectivas simétricas y oblicuas. Asimismo, se aplicaron transformaciones matriciales mediante matrices homogéneas 4x4 y operaciones de cizallamiento para la generación de la proyección gabinete, evidenciando la importancia de las transformaciones matemáticas dentro del proceso de renderizado tridimensional.

El laboratorio también permitió fortalecer conocimientos relacionados con geometría espacial y representación visual en entornos 3D, particularmente en el manejo de sistemas de coordenadas, volúmenes de visión y frustums de proyección. La comparación entre proyecciones ortogonales, gabinete, perspectivas simétricas y perspectivas off-axis permitió comprender cómo las matrices de transformación modifican la percepción de profundidad, el paralelismo de líneas y la ubicación de los puntos de fuga dentro de una escena gráfica.

Desde el punto de vista del diseño de software, la solución fue desarrollada bajo un enfoque modular y estructurado, organizando el sistema en funciones independientes encargadas de configurar cada tipo de proyección, administrar los viewports y renderizar el cubo tridimensional utilizado como objeto de prueba. Esta organización mejora considerablemente la claridad, mantenibilidad y reutilización del código, alineándose con principios fundamentales de ingeniería de software y buenas prácticas de desarrollo de aplicaciones gráficas.

Adicionalmente, la implementación de múltiples viewports permitió visualizar simultáneamente las diferentes configuraciones de proyección dentro de una única ventana OpenGL, facilitando el análisis comparativo entre las representaciones paralelas y perspectivas. El uso del buffer de profundidad (depth testing), junto con las transformaciones de rotación y traslación aplicadas al cubo, contribuyó a mejorar la percepción espacial de la escena y la correcta visualización de las caras tridimensionales del objeto.

Otro aspecto relevante del laboratorio fue la implementación del callback reshape para el redimensionamiento dinámico de la ventana, garantizando la actualización adecuada de los viewports y permitiendo conservar la organización visual de las proyecciones durante la ejecución de la aplicación. Asimismo, el sistema de renderizado de texto implementado mediante TextRenderer permitió identificar claramente cada cuadrante y mejorar la interpretación visual de los resultados obtenidos.

Finalmente, el programa desarrollado compila y ejecuta correctamente utilizando Java 8, JOGL y Apache NetBeans IDE 28 sobre sistemas Windows, demostrando estabilidad funcional y compatibilidad con el pipeline clásico de OpenGL. En conjunto, este laboratorio constituye una base sólida para futuros desarrollos relacionados con gráficos tridimensionales avanzados, shaders GLSL, motores gráficos, simulación computacional y aplicaciones de realidad virtual, consolidando competencias fundamentales en programación gráfica, visualización computacional e ingeniería informática.

BIBLIOGRAFÍA

A continuación, se presentan las principales fuentes bibliográficas, técnicas y académicas utilizadas como referencia para el desarrollo del presente laboratorio de Informática Gráfica y Visualización orientado a la implementación y análisis de diferentes técnicas de proyección tridimensional utilizando OpenGL y JOGL en lenguaje Java.

- Material académico de la asignatura Informática Gráfica y Visualización (COLGII) - PER 15746 - enero 2026.
- Clases virtuales, orientaciones técnicas y fundamentos teóricos proporcionados por el docente Rogerio Orlando Beltrán Castro.
- OpenGL Programming Guide. Guía oficial de programación en OpenGL utilizada como referencia para el manejo del pipeline gráfico, matrices de transformación, renderizado y configuración de proyecciones tridimensionales.
- OpenGL SuperBible. Referencia técnica empleada para comprender conceptos relacionados con renderizado gráfico, geometría tridimensional y programación visual en OpenGL.
- Khronos Group. Documentación oficial de OpenGL y especificaciones técnicas utilizadas para la comprensión de funciones gráficas, frustums de visión y transformaciones matriciales.
- OpenGL. Biblioteca gráfica utilizada para la implementación del sistema de proyecciones tridimensionales y renderizado de la escena 3D.
- JOGL. Biblioteca utilizada para integrar OpenGL dentro de aplicaciones Java y gestionar el contexto gráfico del laboratorio.
- Apache NetBeans. Entorno de desarrollo utilizado para la implementación, compilación y ejecución del proyecto.
- Oracle Corporation. Documentación oficial de Java SE 8 utilizada como referencia para el manejo de eventos, estructura orientada a objetos y administración del entorno de ejecución.
- Computer Graphics: Principles and Practice. Obra de referencia fundamental en el área de gráficos computacionales y visualización tridimensional.
- Computer Graphics with OpenGL. Referencia académica utilizada para fundamentos de informática gráfica, geometría computacional y técnicas de proyección.
- Interactive Computer Graphics. Material de apoyo para comprender transformaciones geométricas, sistemas de coordenadas y renderizado tridimensional en OpenGL.

- Documentación técnica relacionada con matrices homogéneas, transformaciones afines, proyecciones paralelas y perspectivas utilizadas en gráficos computacionales y visualización 3D.
- Recursos académicos y documentación matemática sobre álgebra lineal, trigonometría y geometría espacial aplicadas al cálculo de transformaciones y volúmenes de visión en OpenGL.
- OpenGL Documentation
- JOGL Official Project
- Apache NetBeans Official Website
- Java SE 8 Documentation

AGRADECIMIENTO

Finalmente, deseo expresar mi más sincero agradecimiento al profesor Rogerio Orlando Beltrán Castro por los conocimientos, la orientación y el acompañamiento brindados durante el desarrollo de este laboratorio. Sus explicaciones y aportes en el área de informática gráfica y visualización computacional fueron fundamentales para comprender los principios relacionados con las transformaciones geométricas, las técnicas de proyección tridimensional y el funcionamiento del pipeline gráfico en OpenGL.

Gracias a los conceptos compartidos durante las clases, fue posible desarrollar esta actividad de manera organizada, estructurada y técnicamente fundamentada, aplicando principios relacionados con geometría espacial, matrices homogéneas, volúmenes de visión y representación tridimensional mediante OpenGL y JOGL. Asimismo, las orientaciones proporcionadas permitieron fortalecer la comprensión de las diferencias entre proyecciones paralelas y perspectivas, así como la importancia matemática y visual de las transformaciones de proyección dentro de los gráficos computacionales.

De igual manera, esta experiencia contribuyó significativamente al fortalecimiento de mis competencias en programación gráfica, visualización computacional y desarrollo de software orientado a gráficos tridimensionales, especialmente en aspectos relacionados con modularidad, mantenibilidad, organización estructurada del código y administración de contextos gráficos en aplicaciones Java basadas en OpenGL.

Sin duda, el desarrollo de este laboratorio representó una experiencia académica enriquecedora que permitió consolidar conocimientos fundamentales en informática gráfica y visualización tridimensional, estableciendo una base sólida para futuros proyectos relacionados con gráficos computacionales avanzados, motores de renderizado, simulación interactiva, realidad virtual y desarrollo de aplicaciones gráficas modernas basadas en OpenGL.

¡¡¡Muchas gracias respetado profesor por el acompañamiento y los conocimientos compartidos durante el desarrollo de esta actividad académica!!!

Respetuosamente,

Alejandro De Mendoza